

- Description

This module manages a smart well water level monitoring system, ensuring that:

1. **The water level in the well is continuously monitored to maintain stable water supply.**
2. **If the water level drops below the minimum threshold, the pump is activated to extract water.**
3. **If the water level rises above the maximum threshold, the pump is deactivated to prevent overflow.**
4. **A log of all water level readings, pump activations, and shutdowns is maintained for water resource management.**

This system helps optimize water usage, prevent shortages, and improve automation in water extraction.

- Specification

module SmartWellWaterControlEx

var currentWaterLevel, maxWaterLevel, minWaterLevel: real

var pumpActive: boolean

process updateWaterLevel(changeAmount: real)

functional scenario form:

currentWaterLevel <= maxWaterLevel && pumpActive = pumpActive, Water level maintained.

||

currentWaterLevel < minWaterLevel && pumpActive = true, Water pump activated.

||

currentWaterLevel > maxWaterLevel && pumpActive = false, Water pump deactivated.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
```

```

import java.util.*;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c114_SmartWellSystem - All-in-one Demo
 * Features:
 * - Core SmartWell logic (water level control with hysteresis)
 * - Sensors (water level, water quality, temperature)
 * - Pump controller & energy usage estimation
 * - Alert system & file-backed logger (append mode)
 * - Swing GUI (real-time values, charts-lite table, actions)
 * - Simulation engine (rain/evaporation/leak/noise) with thread
 * - History replay (log playback)
 * - CLI fallback testcases (VT/FT/Z3/UVT)
 * Compile: javac code/c114_SmartWellSystem.java
 * Run      : java code.c114_SmartWellSystem
 * =====
 */
/* ===== Utilities ===== */
class Str {
    static String fmt2(double v) { return new DecimalFormat("0.00").format(v); }
    static String now() { return LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
}
/* ===== Logger ===== */
class DataLogger {
    private final File file;
    private final ConcurrentLinkedQueue<String> inMemory = new ConcurrentLinkedQueue<>();
    private final int inMemoryCap;
    DataLogger(String path, int inMemoryCap) {
        this.file = new File(path);
        this.inMemoryCap = Math.max(50, inMemoryCap);
        try {
            if (!file.exists()) {
                File parent = file.getParentFile();
                if (parent != null) parent.mkdirs();
                file.createNewFile();
            }
        } catch (IOException e) {
            System.err.println("[Logger] Failed to create log file: " + e.getMessage());
        }
    }
}

```

```

    }
}
synchronized void log(String message) {
    String line = "[" + Str.now() + "]" + message;
    // memory
    inMemory.add(line);
    while (inMemory.size() > inMemoryCap) inMemory.poll();
    // file
    try (FileOutputStream fos = new FileOutputStream(file, true);
        OutputStreamWriter osw = new OutputStreamWriter(fos, StandardCharsets.UTF_8);
        BufferedWriter bw = new BufferedWriter(osw)) {
        bw.write(line);
        bw.newLine();
    } catch (IOException e) {
        System.err.println("[Logger] Write failed: " + e.getMessage());
    }
}
List<String> recent(int max) {
    ArrayList<String> list = new ArrayList<>(inMemory);
    if (list.size() <= max) return list;
    return list.subList(list.size() - max, list.size());
}
File getFile() { return file; }
}
/* ===== Alerts ===== */
class AlertSystem {
    enum Severity { INFO, WARNING, CRITICAL }
    static class Alert {
        final Severity severity;
        final String message;
        final LocalDateTime time = LocalDateTime.now();
        Alert(Severity s, String m) { this.severity = s; this.message = m; }
        @Override public String toString() {
            return "[" + time + "]" + severity + " " + message;
        }
    }
    private final List<Alert> alerts = new ArrayList<>();
    private final DataLogger logger;
    AlertSystem(DataLogger logger) { this.logger = logger; }
    void push(AlertSystem.Severity s, String msg) {
        Alert a = new Alert(s, msg);
    }
}

```

```

        alerts.add(a);
        logger.log("ALERT " + s + " :: " + msg);
        // For demo, print to console
        System.out.println(a);
        if (s == Severity.CRITICAL) {
            // hook for future: email/SMS/etc.
        }
    }
    List<Alert> all() { return Collections.unmodifiableList(alerts); }
    List<Alert> recent(int n) {
        if (alerts.size() <= n) return new ArrayList<>(alerts);
        return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
    }
}
/* ===== Sensors ===== */
abstract class Sensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    Sensor(String name) { this.name = name; }
    abstract void update(SmartWellContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}
class SmartWellContext {
    public double currentWaterLevel; // meters
    public double minWaterLevel;
    public double maxWaterLevel;
    public boolean pumpActive;
    public double waterTemperature; // °C
    public double ph; // 0-14
    public double turbidity; // NTU
    public double conductivity; // µS/cm
    SmartWellContext cloneCopy() {
        SmartWellContext c = new SmartWellContext();
        c.currentWaterLevel = currentWaterLevel;
        c.minWaterLevel = minWaterLevel;
        c.maxWaterLevel = maxWaterLevel;
        c.pumpActive = pumpActive;
    }
}

```

```

        c.waterTemperature = waterTemperature;
        c.ph = ph;
        c.turbidity = turbidity;
        c.conductivity = conductivity;
        return c;
    }
}

class WaterLevelSensor extends Sensor {
    private double drift = 0; // simulate bias
    WaterLevelSensor() { super("WaterLevelSensor"); }
    @Override
    void update(SmartWellContext ctx) {
        // simulate tiny noise and slow drift
        drift += (rnd.nextDouble() - 0.5) * 0.001;
        value = ctx.currentWaterLevel + drift + (rnd.nextDouble() - 0.5
    ) * 0.01;
        if (value < 0) { value = 0; }
        ok = true;
        status = "OK";
        // occasionally introduce glitch
        if (rnd.nextDouble() < 0.0005) {
            ok = false; status = "GLITCH"; value = -1;
        }
    }
}

class TemperatureSensor extends Sensor {
    TemperatureSensor() { super("TemperatureSensor"); }
    @Override
    void update(SmartWellContext ctx) {
        // add slight noise
        value = ctx.waterTemperature + (rnd.nextDouble() - 0.5) * 0.2;
        ok = (value > -5 && value < 80);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class WaterQualitySensor extends Sensor {
    private double ph;
    private double turbidity;
    private double conductivity;
    WaterQualitySensor() { super("WaterQualitySensor"); }
    @Override
    void update(SmartWellContext ctx) {
        // store combined metrics in fields; expose one as 'value' for
table

```

```

        ph = clamp(ctx.ph + (rnd.nextDouble() - 0.5) * 0.05, 0, 14);
        turbidity = Math.max(0, ctx.turbidity + (rnd.nextDouble() - 0.5) * 0.5);
        conductivity = Math.max(0, ctx.conductivity + (rnd.nextDouble() - 0.5) * 5.0);
        // encode: value = weighted quality index (lower is better for
        turbidity, moderate for conductivity, neutral for ph around 7)
        double phScore = Math.abs(ph - 7.0);           // lower is better
        double turbScore = turbidity / 5.0;           // ~0-10 NTU -> 0-2
        double condScore = Math.max(0, (conductivity - 200.0) / 400.0);
        // >200 µS/h penalize
        value = phScore + turbScore + condScore;
        ok = (ph >= 6.0 && ph <= 8.5) && turbidity <= 10.0 && conductivity <= 800.0;
        status = ok ? "OK" : "QUALITY_ISSUE";
    }
    double getPh() { return ph; }
    double getTurbidity() { return turbidity; }
    double getConductivity() { return conductivity; }
    private static double clamp(double v, double lo, double hi) {
        return Math.max(lo, Math.min(hi, v));
    }
}

/* ===== Pump Controller =====
*/

//class PumpController {
//    private boolean active = false;
//    private double onThreshold; // turn ON when below
//    private double offThreshold; // turn OFF when above
//    private double kwRating = 0.75; // 0.75 kW
//    private double energyKWh = 0.0;
//
//    PumpController(double onThreshold, double offThreshold) {
//        // ensure hysteresis
//        this.onThreshold = Math.min(onThreshold, offThreshold - 0.1);
//        this.offThreshold = Math.max(offThreshold, onThreshold + 0.1);
//    }
//
//    void update(double currentWaterLevel) {
//        if (currentWaterLevel < onThreshold) active = true;
//        else if (currentWaterLevel > offThreshold) active = false;
//    }

```

```

//
//// void addTimeStepEnergy(double seconds) {
////     if (active) {
////         energyKWh += kWRating * (seconds / 3600.0);
////     }
//// }
//
// boolean isActive() { return active; }
// void forceOff() { active = false; }
// void forceOn() { active = true; }
//
// double getEnergyKWh() { return energyKWh; }
// void resetEnergy() { energyKWh = 0.0; }
//
// double getOnThreshold() { return onThreshold; }
// double getOffThreshold() { return offThreshold; }
//
// void setThresholds(double onTh, double offTh) {
//     this.onThreshold = Math.min(onTh, offTh - 0.1);
//     this.offThreshold = Math.max(offTh, onTh + 0.1);
// }
//}
class PumpController {
    private boolean active = false;
    private double onThreshold; // turn ON when below
    private double offThreshold; // turn OFF when above
    private double kWRating = 0.75; // 0.75 kW
    private double energyKWh = 0.0;
    PumpController(double onThreshold, double offThreshold) {
        // ensure hysteresis
        this.onThreshold = Math.min(onThreshold, offThreshold - 0.1);
        this.offThreshold = Math.max(offThreshold, onThreshold + 0.1);
    }
    void update(double currentWaterLevel) {
        if (currentWaterLevel < onThreshold) active = true;
        else if (currentWaterLevel > offThreshold) active = false;
    }
    // void addTimeStepEnergy(double seconds) {
    //     if (active) {
    //         energyKWh += kWRating * (seconds / 3600.0);
    //     }
    // }
    // }
    boolean isActive() { return active; }
    void forceOff() { active = false; }

```

```

    void forceOn() { active = true; }
    double getEnergyKWh() { return energyKWh; }
    void resetEnergy() { energyKWh = 0.0; }
    double getOnThreshold() { return onThreshold; }
    double getOffThreshold() { return offThreshold; }
    void setThresholds(double onTh, double offTh) {
        this.onThreshold = Math.min(onTh, offTh - 0.1);
        this.offThreshold = Math.max(offTh, onTh + 0.1);
    }
}

/* ===== SmartWell core ===== */
/

class SmartWell {
    private final DataLogger logger;
    private final AlertSystem alert;
    // bounds
    private double maxWaterLevel;
    private double minWaterLevel;
    private double currentWaterLevel;
    // environmental state
    private double waterTemperature; // °C
    private double ph;
    private double turbidity; // NTU
    private double conductivity; // µS/cm
    // components
    private final PumpController pump;
    private final WaterLevelSensor levelSensor = new WaterLevelSensor()
;
    private final TemperatureSensor tempSensor = new TemperatureSensor(
);
    private final WaterQualitySensor qualitySensor = new WaterQualitySe
nsor();
    // calibration & maintenance
    private boolean selfCleaning = false;
    private int cleaningTicks = 0;
    // listeners/UI
    private final List<Runnable> listeners = new ArrayList<>();
    SmartWell(double maxWaterLevel, double minWaterLevel, double initia
lWaterLevel,
        DataLogger logger, AlertSystem alert) {
        this.maxWaterLevel = maxWaterLevel;
        this.minWaterLevel = minWaterLevel;
        this.currentWaterLevel = initialWaterLevel;
        this.logger = logger;

```

```

        this.alert = alert;
        // default env
        this.waterTemperature = 18.0;
        this.ph = 7.2;
        this.turbidity = 2.0;
        this.conductivity = 250.0;
        this.pump = new PumpController(minWaterLevel, maxWaterLevel);
        logger.log("SmartWell initialized. Level = " + Str.fmt2(current
WaterLevel)
                    + "m (min=" + minWaterLevel + ", max=" + maxWaterLevel
+ ")");
    }
    void addListener(Runnable r) { listeners.add(r); }
    private void emit() { for (Runnable r : listeners) try { r.run(); }
catch (Throwable ignored) {} }
    /* ===== Public API (was: updateWaterLevel) ===== */
    public synchronized void changeWaterLevel(double delta) {
        currentWaterLevel += delta;
        if (currentWaterLevel < 0) currentWaterLevel = 0;
        managePumpControl(); // pump may turn on/off
        sampleSensors();
        // UI/Log
        String msg = "Current Water Level: " + Str.fmt2(currentWaterLev
el) + " meters | Pump: " + (pump.isActive() ? "ON" : "OFF");
        logger.log(msg);
        System.out.println(msg);
        emit();
    }
    private void managePumpControl() {
        pump.update(currentWaterLevel);
        String s = pump.isActive() ? "Water Pump Activated." : "Water P
ump Deactivated.";
        logger.log(s);
        // Alerts based on bounds:
        if (currentWaterLevel < minWaterLevel) {
            alert.push(AlertSystem.Severity.WARNING, "Level below MIN (
" + Str.fmt2(currentWaterLevel) + "m < " + Str.fmt2(minWaterLevel) + "m
)");
        }
        if (currentWaterLevel > maxWaterLevel) {
            alert.push(AlertSystem.Severity.WARNING, "Level above MAX (
" + Str.fmt2(currentWaterLevel) + "m > " + Str.fmt2(maxWaterLevel) + "m
)");
        }
    }
}

```

```

    }
    private void sampleSensors() {
        SmartWellContext ctx = buildContext();
        levelSensor.update(ctx);
        tempSensor.update(ctx);
        qualitySensor.update(ctx);
        // quality alarms
        if (!qualitySensor.isOk()) {
            alert.push(AlertSystem.Severity.WARNING,
                "Water quality issue: pH=" + Str.fmt2(qualitySensor
.getPh())
                + ", turbidity=" + Str.fmt2(qualitySensor.g
etTurbidity())
                + ", conductivity=" + Str.fmt2(qualitySenso
r.getConductivity()));
        }
        // sensor glitch
        if (!levelSensor.isOk()) {
            alert.push(AlertSystem.Severity.CRITICAL, "Level sensor gli
tch detected.");
        }
    }
    SmartWellContext buildContext() {
        SmartWellContext c = new SmartWellContext();
        c.currentWaterLevel = currentWaterLevel;
        c.minWaterLevel = minWaterLevel;
        c.maxWaterLevel = maxWaterLevel;
        c.pumpActive = pump.isActive();
        c.waterTemperature = waterTemperature;
        c.ph = ph;
        c.turbidity = turbidity;
        c.conductivity = conductivity;
        return c;
    }
    /* ===== Simulation hooks ===== */
    synchronized void tickSeconds(double seconds) {
        // pump energy consumption
        pump.addTimeStepEnergy(seconds);
        // self-cleaning routine (affects turbidity slightly)
        if (selfCleaning) {
            cleaningTicks++;
            turbidity = Math.max(0.5, turbidity - 0.02);
            if (cleaningTicks >= 50) {
                selfCleaning = false;
            }
        }
    }
}

```

```

        cleaningTicks = 0;
        logger.log("Self-cleaning finished.");
    }
}
sampleSensors();
emit();
}
synchronized void applyEnvironment(double rainDelta, double evapDelta, double leakDelta) {
    // +- deltas are in meters per step
    currentWaterLevel += rainDelta;
    currentWaterLevel -= evapDelta;
    currentWaterLevel -= leakDelta;
    if (currentWaterLevel < 0) currentWaterLevel = 0;
    // temp & quality drift (rough model)
    waterTemperature += (rainDelta > 0 ? -
0.05 : 0.02); // rain cools slightly
    waterTemperature = clamp(waterTemperature, 0, 50);
    // rainfall reduces conductivity & turbidity slightly (dilution
), evaporation increases
    conductivity = clamp(conductivity + (-
rainDelta * 5.0) + (evapDelta * 3.0), 50, 1200);
    turbidity = clamp(turbidity + (-
rainDelta * 0.5) + (evapDelta * 0.4), 0, 100);
    ph = clamp(ph + (rainDelta > 0 ? -0.01 : 0.005), 5.0, 9.5);
    managePumpControl();
    sampleSensors();
    logger.log(String.format(Locale.US,
        "Env step: rain=%.3f, evap=%.3f, leak=%.3f | level=%.2f
, T=%.1f°C",
        rainDelta, evapDelta, leakDelta, currentWaterLevel, waterTemperature));
    emit();
}
/* ===== Maintenance ===== */
synchronized void startSelfCleaning() {
    if (!selfCleaning) {
        selfCleaning = true;
        cleaningTicks = 0;
        logger.log("Self-cleaning started.");
    }
}
synchronized void calibrateLevel(double observedMeters) {

```

```

        // naive calibration: align current level to observed (simulate
        recalibration)
        double old = currentWaterLevel;
        currentWaterLevel = Math.max(0, observedMeters);
        logger.log("Calibration: level " + Str.fmt2(old) + " -> " + Str
.fmt2(currentWaterLevel));
        sampleSensors();
        emit();
    }
    synchronized void setThresholds(double newMin, double newMax) {
        double oldMin = minWaterLevel, oldMax = maxWaterLevel;
        minWaterLevel = Math.max(0, Math.min(newMin, newMax - 0.2));
        maxWaterLevel = Math.max(minWaterLevel + 0.2, newMax);
        pump.setThresholds(minWaterLevel, maxWaterLevel);
        logger.log("Thresholds changed: min " + Str.fmt2(oldMin) + "->"
+ Str.fmt2(minWaterLevel) +
            ", max " + Str.fmt2(oldMax) + "->" + Str.fmt2(maxWaterL
evel));
        sampleSensors();
        emit();
    }
    /* ===== Getters for GUI ===== */
    // double getCurrentWaterLevel() { return currentWaterLevel; }
    // double getMinWaterLevel() { return minWaterLevel; }
    // double getMaxWaterLevel() { return maxWaterLevel; }
    // double getWaterTemperature() { return waterTemperature; }
    // double getPh() { return ph; }
    // double getTurbidity() { return turbidity; }
    // double getConductivity() { return conductivity; }
    //
    // boolean isPumpActive() { return pump.isActive(); }
    // double getPumpEnergyKWh() { return pump.getEnergyKWh(); }
    //
    // WaterLevelSensor getLevelSensor() { return levelSensor; }
    // TemperatureSensor getTempSensor() { return tempSensor; }
    // WaterQualitySensor getQualitySensor() { return qualitySensor; }
    double getCurrentWaterLevel() { return currentWaterLevel; }
    double getMinWaterLevel() { return minWaterLevel; }
    double getMaxWaterLevel() { return maxWaterLevel; }
    double getWaterTemperature() { return waterTemperature; }
    double getPh() { return ph; }
    double getTurbidity() { return turbidity; }
    double getConductivity() { return conductivity; }
    boolean isPumpActive() { return pump.isActive(); }

```

```

    double getPumpEnergyKWh() { return pump.getEnergyKWh(); }
    WaterLevelSensor getLevelSensor() { return levelSensor; }
    TemperatureSensor getTempSensor() { return tempSensor; }
    WaterQualitySensor getQualitySensor() { return qualitySensor; }
    /* ===== Helpers ===== */
    private static double clamp(double v, double lo, double hi) { return
n Math.max(lo, Math.min(hi, v)); }
}
/* ===== Simulation Engine =====
= */
class SimulationEngine implements Runnable {
    private final SmartWell well;
    private final DataLogger logger;
    private final AtomicBoolean running = new AtomicBoolean(false);
    private Thread thread;
    // parameters
    private volatile double rainMean = 0.004;    // meters per tick (po
sitive adds water)
    private volatile double evapMean = 0.002;    // meters per tick (po
sitive removes water)
    private volatile double leakMean = 0.0005;    // meters per tick (po
sitive removes water)
    private volatile double tickSeconds = 1.0;    // real-time step
    private volatile long sleepMillis = 250L;    // wall-
clock sleep (4 ticks per second)
    private volatile boolean randomize = true;
    private final Random rnd = new Random();
    SimulationEngine(SmartWell well, DataLogger logger) {
        this.well = well;
        this.logger = logger;
    }
    void start() {
        if (running.get()) return;
        running.set(true);
        thread = new Thread(this, "SimulationEngine");
        thread.setDaemon(true);
        thread.start();
        logger.log("Simulation started.");
    }
    void stop() {
        running.set(false);
        if (thread != null) {
            try { thread.join(2000); } catch (InterruptedException igno
red) { }

```

```

    }
    logger.log("Simulation stopped.");
}
boolean isRunning() { return running.get(); }
void setParams(double rainMean, double evapMean, double leakMean, double tickSeconds, long sleepMillis, boolean randomize) {
    this.rainMean = Math.max(0, rainMean);
    this.evapMean = Math.max(0, evapMean);
    this.leakMean = Math.max(0, leakMean);
    this.tickSeconds = Math.max(0.1, tickSeconds);
    this.sleepMillis = Math.max(20, sleepMillis);
    this.randomize = randomize;
    logger.log(String.format(Locale.US,
        "Sim params updated: rain=%.4f, evap=%.4f, leak=%.4f, dt=%.2fs, delay=%dms, random=%s",
        this.rainMean, this.evapMean, this.leakMean, this.tickSeconds, this.sleepMillis, String.valueOf(this.randomize)));
}
@Override
public void run() {
    while (running.get()) {
        double rain = rainMean;
        double evap = evapMean;
        double leak = leakMean;
        if (randomize) {
            rain = Math.max(0, rain + (rnd.nextDouble() - 0.5) * rainMean * 0.8);
            evap = Math.max(0, evap + (rnd.nextDouble() - 0.5) * evapMean * 0.8);
            leak = Math.max(0, leak + (rnd.nextDouble() - 0.5) * leakMean * 1.0);
        }
        well.applyEnvironment(rain, evap, leak);
        well.tickSeconds(tickSeconds);
        try {
            Thread.sleep(sleepMillis);
        } catch (InterruptedException e) {
            // graceful stop
            break;
        }
    }
}
}
}

```

```

/* ===== Replay (History) =====
*/
class ReplayEngine {
    private final SmartWell well;
    private final DataLogger logger;
    ReplayEngine(SmartWell well, DataLogger logger) {
        this.well = well;
        this.logger = logger;
    }
    /**
     * Very simple parser: this looks for "Env step: rain=..., evap=...
, leak=..." lines
     * and replays them with a small delay.
     */
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
        logger.log("Replay started from: " + logFile.getAbsolutePath())
;
        try (BufferedReader br = new BufferedReader(new InputSteamRead
er(new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.contains("Env step:")) continue;
                // parse numbers
                try {
                    // example: Env step: rain=0.003, evap=0.001, Leak=
0.000 | Level=...

                    int idx = line.indexOf("Env step:");
                    String seg = line.substring(idx);
                    String[] parts = seg.split("\\|")[0].split(",");
                    double rain = Double.parseDouble(parts[0].split("=")
)[1].trim());
                    double evap = Double.parseDouble(parts[1].split("=")
)[1].trim());
                    double leak = Double.parseDouble(parts[2].split("=")
)[1].trim());

                    well.applyEnvironment(rain, evap, leak);
                    well.tickSeconds(1.0);
                    Thread.sleep(delayMs);
                } catch (Throwable ignored) { }
            }
        }
    }
}

```

```

        } catch (IOException e) {
            System.err.println("[Replay] Error: " + e.getMessage());
        }
        logger.log("Replay finished.");
    }
}

/* ===== GUI ===== */
class SmartWellGUI extends JFrame {
    private final SmartWell well;
    private final DataLogger logger;
    private final AlertSystem alerts;
    private final SimulationEngine engine;
    private final ReplayEngine replay;
    // labels
    private JLabel lblLevel, lblMin, lblMax, lblPump, lblTemp, lblPh, lblTurb, lblCond, lblEnergy;
    private JTextArea txtLog;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JSpinner spMin, spMax;
    private JSpinner spRain, spEvap, spLeak, spDt, spDelay;
    private JCheckBox cbRandomize;
    private JButton btnStart, btnStop, btnClean, btnCalib, btnAdd, btnRemove, btnReplay;
    private javax.swing.Timer uiTimer;

    SmartWellGUI(SmartWell well, DataLogger logger, AlertSystem alerts, SimulationEngine engine, ReplayEngine replay) {
        super("SmartWell - Intelligent Well Control System");
        this.well = well;
        this.logger = logger;
        this.alerts = alerts;
        this.engine = engine;
        this.replay = replay;
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        buildUi();
        setSize(1100, 720);
        setLocationRelativeTo(null);
        // update UI periodically
        uiTimer = new javax.swing.Timer(500, e -> refreshUi());
        uiTimer.start();
        // register well listener for immediate refresh
        well.addListener(this::refreshUi);
    }

    private JPanel titled(String title, JComponent inner) {

```

```

        JPanel p = new JPanel(new BorderLayout());
        p.setBorder(BorderFactory.createTitledBorder(title));
        p.add(inner, BorderLayout.CENTER);
        return p;
    }
    private void buildUi() {
        JPanel root = new JPanel(new BorderLayout(10, 10));
        root.setBorder(new EmptyBorder(10,10,10,10));
        setContentPane(root);
        /* ===== LEFT: Status & Controls ===== */
        JPanel left = new JPanel();
        left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
        // status grid
        JPanel status = new JPanel(new GridLayout(0,2,6,6));
        lblLevel = new JLabel("-");
        lblMin = new JLabel("-");
        lblMax = new JLabel("-");
        lblPump = new JLabel("-");
        lblTemp = new JLabel("-");
        lblPh = new JLabel("-");
        lblTurb = new JLabel("-");
        lblCond = new JLabel("-");
        lblEnergy = new JLabel("-");
        status.add(new JLabel("Level (m):")); status.add(lblLevel);
        status.add(new JLabel("Min (m):")); status.add(lblMin);
        status.add(new JLabel("Max (m):")); status.add(lblMax);
        status.add(new JLabel("Pump:")); status.add(lblPump);
        status.add(new JLabel("Temp (°C):")); status.add(lblTemp);
        status.add(new JLabel("pH:")); status.add(lblPh);
        status.add(new JLabel("Turbidity:")); status.add(lblTurb);
        status.add(new JLabel("Conductivity:")); status.add(lblCond);
        status.add(new JLabel("Energy (kWh):")); status.add(lblEnergy);
        left.add(titled("Live Status", status));
        // thresholds
        JPanel th = new JPanel(new GridLayout(0,2,6,6));
        spMin = new JSpinner(new SpinnerNumberModel(3.0, 0.0, 100.0, 0.
1));
        spMax = new JSpinner(new SpinnerNumberModel(10.0, 0.0, 100.0, 0
.1));
        JButton btnApplyTh = new JButton("Apply Thresholds");
        btnApplyTh.addActionListener(e -> {
            double mn = ((Number)spMin.getValue()).doubleValue();
            double mx = ((Number)spMax.getValue()).doubleValue();
            well.setThresholds(mn, mx);
        });
    }
}

```

```

    });
    th.add(new JLabel("Min level (m)")); th.add(spMin);
    th.add(new JLabel("Max level (m)")); th.add(spMax);
    th.add(new JLabel(" ")); th.add(btnApplyTh);
    left.add(Box.createVerticalStrut(6));
    left.add(titled("Thresholds", th));
    // operations
    JPanel ops = new JPanel(new GridLayout(0,2,6,6));
    btnAdd = new JButton("Add Water +0.5m");
    btnRemove = new JButton("Remove Water -0.5m");
    btnClean = new JButton("Start Self-Cleaning");
    btnCalib = new JButton("Calibrate Level (observe)");
    btnAdd.addActionListener(e -> well.changeWaterLevel(+0.5));
    btnRemove.addActionListener(e -> well.changeWaterLevel(-0.5));
    btnClean.addActionListener(e -> well.startSelfCleaning());
    btnCalib.addActionListener(e -> {
        String s = JOptionPane.showInputDialog(this, "Enter observe
d level (m):", Str.fmt2(well.getCurrentWaterLevel()));
        if (s == null) return;
        try {
            double v = Double.parseDouble(s);
            well.calibrateLevel(v);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Invalid number.");
        }
    });
    ops.add(btnAdd); ops.add(btnRemove);
    ops.add(btnClean); ops.add(btnCalib);
    left.add(Box.createVerticalStrut(6));
    left.add(titled("Operations", ops));
    // sim controls
    //      JPanel sim = new JPanel(new GridLayout(0,2,6,6));
    //      spRain = new JSpinner(new SpinnerNumberModel(0.004, 0.0, 1.0,
0.001));
    //      spEvap = new JSpinner(new SpinnerNumberModel(0.002, 0.0, 1.0,
0.001));
    //      spLeak = new JSpinner(new SpinnerNumberModel(0.0005, 0.0, 1.0,
0.0001));
    //      spDt   = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0
.1));
    //      spDelay= new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10
));
    //      cbRandomize = new JCheckBox("Randomize", true);
    //      btnStart = new JButton("Start Simulation");

```

```

//      btnStop = new JButton("Stop Simulation");
//      btnReplay = new JButton("Replay from Log");
//      btnStart.addActionListener(e -> engine.setParams(
//          ((Number)spRain.getValue()).doubleValue(),
//          ((Number)spEvap.getValue()).doubleValue(),
//          ((Number)spLeak.getValue()).doubleValue(),
//          ((Number)spDt.getValue()).doubleValue(),
//          ((Number)spDelay.getValue()).longValue(),
//          cbRandomize.isSelected()
//      ));
//      JPanel sim = new JPanel(new GridLayout(0,2,6,6));
//      spRain = new JSpinner(new SpinnerNumberModel(0.004, 0.0, 1.0, 0
//          .001));
//      spEvap = new JSpinner(new SpinnerNumberModel(0.002, 0.0, 1.0, 0
//          .001));
//      spLeak = new JSpinner(new SpinnerNumberModel(0.0005, 0.0, 1.0,
//          0.0001));
//      spDt = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0.1
//          ));
//      spDelay= new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10)
//          );
//      cbRandomize = new JCheckBox("Randomize", true);
//      btnStart = new JButton("Start Simulation");
//      btnStop = new JButton("Stop Simulation");
//      btnReplay = new JButton("Replay from Log");
//      btnStart.addActionListener(e -> engine.setParams(
//          ((Number)spRain.getValue()).doubleValue(),
//          ((Number)spEvap.getValue()).doubleValue(),
//          ((Number)spLeak.getValue()).doubleValue(),
//          ((Number)spDt.getValue()).doubleValue(),
//          ((Number)spDelay.getValue()).longValue(),
//          cbRandomize.isSelected()
//      ));
//
//      btnStart.addActionListener(e -> engine.start());
//      btnStop.addActionListener(e -> engine.stop());
//      btnReplay.addActionListener(e -> new Thread(() -> replay.replay
//          (logger.getFile(), 50), "Replay").start());
//      sim.add(new JLabel("Rain (m/tick)")); sim.add(spRain);
//      sim.add(new JLabel("Evap (m/tick)")); sim.add(spEvap);
//      sim.add(new JLabel("Leak (m/tick)")); sim.add(spLeak);
//      sim.add(new JLabel("Δt (s)"));          sim.add(spDt);
//      sim.add(new JLabel("Delay (ms)"));      sim.add(spDelay);
//      sim.add(new JLabel(" "));              sim.add(cbRandomize);

```

```

sim.add(btnStart);                                sim.add(btnStop);
sim.add(new JLabel(" "));                          sim.add(btnReplay);
left.add(Box.createVerticalStrut(6));
left.add(titled("Simulation", sim));
root.add(left, BorderLayout.WEST);
/* ===== CENTER: Sensors table & Alerts ===== */
JPanel center = new JPanel(new BorderLayout(6,6));
// table
tblModel = new DefaultTableModel(new String[]{"Sensor", "Value",
"OK", "Status"}, 0) {
    @Override public boolean isCellEditable(int row, int column
) { return false; }
};
tblSensors = new JTable(tblModel);
JScrollPane spTbl = new JScrollPane(tblSensors);
center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
// alerts recent table
JTextArea txtAlerts = new JTextArea(8, 60);
txtAlerts.setEditable(false);
center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
root.add(center, BorderLayout.CENTER);
/* ===== RIGHT: Log ===== */
txtLog = new JTextArea();
txtLog.setEditable(false);
JScrollPane spLog = new JScrollPane(txtLog);
root.add(titled("Log (tail)", spLog), BorderLayout.EAST);
// refresh alerts text area
javax.swing.Timer alertsTimer = new javax.swing.Timer(1000, e -
> {
    StringBuilder sb = new StringBuilder();
    for (AlertSystem.Alert a : alerts.recent(15)) {
        sb.append(a).append("\n");
    }
    txtAlerts.setText(sb.toString());
    txtAlerts.setCaretPosition(txtAlerts.getDocument().getLengt
h());
});
alertsTimer.start();
}
private void refreshUi() {
    lblLevel.setText(Str.fmt2(well.getCurrentWaterLevel()));
    lblMin.setText(Str.fmt2(well.getMinWaterLevel()));
    lblMax.setText(Str.fmt2(well.getMaxWaterLevel()));
}

```

```

        lblPump.setText(well.isPumpActive() ? "ON" : "OFF");
        lblPump.setForeground(well.isPumpActive() ? new Color(0,120,0)
: Color.RED);
        lblTemp.setText(Str.fmt2(well.getWaterTemperature()));
        lblPh.setText(Str.fmt2(well.getPh()));
        lblTurb.setText(Str.fmt2(well.getTurbidity()));
        lblCond.setText(Str.fmt2(well.getConductivity()));
        lblEnergy.setText(Str.fmt2(well.getPumpEnergyKWh()));
        // sensors
        tblModel.setRowCount(0);
        tblModel.addRow(new Object[]{
            well.getLevelSensor().getName(),
            Str.fmt2(well.getLevelSensor().getValue()),
            well.getLevelSensor().isOk(), well.getLevelSensor().get
Status()
        });
        tblModel.addRow(new Object[]{
            well.getTempSensor().getName(),
            Str.fmt2(well.getTempSensor().getValue()),
            well.getTempSensor().isOk(), well.getTempSensor().getSt
atus()
        });
        tblModel.addRow(new Object[]{
            well.getQualitySensor().getName(),
            Str.fmt2(well.getQualitySensor().getValue()) + " (QIdx)
",
            well.getQualitySensor().isOk(), well.getQualitySensor().
getStatus()
        });
        // tail log
        List<String> lines = logger.recent(200);
        StringBuilder sb = new StringBuilder();
        for (String s : lines) sb.append(s).append("\n");
        txtLog.setText(sb.toString());
        txtLog.setCaretPosition(txtLog.getDocument().getLength());
    }
}
/* ===== CLI Harness ===== */
class CLIHarness {
    private final SmartWell well;
    CLIHarness(SmartWell well) { this.well = well; }
    void runAll() {
        System.out.println("==== CLI Demo (VT) =====");
        vt();
    }
}

```

```

        System.out.println("==== CLI Demo (FT) =====");
        ft();
        System.out.println("==== CLI Demo (Z3 mimic) =====");
        z3();
        System.out.println("==== CLI Demo (UVT) =====");
        uvt();
    }
    // Your original VT
    void vt() {
        well.changeWaterLevel(-4.0);
        well.changeWaterLevel(5.0);
        well.changeWaterLevel(4.0);
        well.changeWaterLevel(-2.0);
    }
    // FT / Edge-ish
    void ft() {
        well.changeWaterLevel(0);
        well.changeWaterLevel(0); // will clamp to 0
        well.changeWaterLevel(0);
        well.changeWaterLevel(1.7976931348623157E308); // ridiculously
big
    }
    void z3() {
        well.changeWaterLevel(-4.0);
        well.changeWaterLevel(5.0);
        well.changeWaterLevel(4.0);
        well.changeWaterLevel(-2.0);
    }
    void uvt() {
        well.changeWaterLevel(-4.0);
        well.changeWaterLevel(5.0);
        well.changeWaterLevel(4.0);
        well.changeWaterLevel(-2.0);
    }
}
/* ===== Main ===== */
public class c114_SmartWellSystemEx {
    public static void main(String[] args) {
        // Decide GUI or CLI by presence of a headful environment, or c
ommand-line flag

        //         boolean forceCli = Arrays.asList(args).contains("--cli");
        boolean forceCli = true;
        // logger & alerts

```

```

        DataLogger logger = new DataLogger("well_log.txt", 500);
        AlertSystem alertSystem = new AlertSystem(logger);
        // SmartWell (initial thresholds & state aligned to your origin
al)

        SmartWell well = new SmartWell(10.0, 3.0, 6.0, logger, alertSys
tem);

        // simulation & replay
        SimulationEngine engine = new SimulationEngine(well, logger);
        ReplayEngine replay = new ReplayEngine(well, logger);

        // ===== CLI
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            CLIHarness harness = new CLIHarness(well);
            harness.vt();
            //      harness.ft();
            //      harness.z3();
            //      harness.uvt();
            for (int i = 0; i < 20; i++) {
                well.applyEnvironment(0.004, 0.002, 0.0006);
                well.tickSeconds(1.0);
                try { Thread.sleep(100L); } catch (Exception ignored) {
            }

                }
            logger.log("CLI demo finished.");
            System.out.println("CLI demo finished. Log written to: " +
logger.getFile().getAbsolutePath());
            return;
        }
        else {
            // ===== GUI path =====
            logger.log("Running in GUI mode.");
            SwingUtilities.invokeLater(() -> {
                SmartWellGUI gui = new SmartWellGUI(well, logger, alert
System, engine, replay);
                gui.setVisible(true);
            });
        }
    }

    private static boolean headless() {
        try {
            return GraphicsEnvironment.isHeadless();
        } catch (Throwable t) {
            return true;
        }
    }

```

```

    }
}
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c114_SmartWellSystemEx.java	37.2 %	35	59	94
> AlertSystem	0.0 %	0	2	2
> ReplayEngine	0.0 %	0	8	8
> SimulationEngine	0.0 %	0	8	8
> DataLogger	25.0 %	2	6	8
> c114_SmartWellSystemEx	50.0 %	3	3	6
> TemperatureSensor	50.0 %	3	3	6
> WaterLevelSensor	50.0 %	2	2	4
> WaterQualitySensor	50.0 %	5	5	10
▼ SmartWell	57.1 %	16	12	28
▶ tickSeconds(double)	25.0 %	1	3	4
▶ applyEnvironment(double, double, d	50.0 %	3	3	6
▶ emit()	50.0 %	1	1	2
▶ sampleSensors()	50.0 %	2	2	4
▶ changeWaterLevel(double)	75.0 %	3	1	4
▶ managePumpControl()	100.0 %	6	0	6
▶ clamp(double, double, double)		0	0	0
▶ SmartWell(double, double, double, C		0	0	0
▶ buildContext()		0	0	0
> PumpController	100.0 %	4	0	4
> CLIHarness		0	0	0
> Sensor		0	0	0
> SmartWellContext		0	0	0
> Str		0	0	0

3. Vibration testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c114_SmartWellSystemEx.java	32.7 %	964	1,988	2,952
> SmartWellContext	7.3 %	3	38	41
> ReplayEngine	7.4 %	9	113	122
> SimulationEngine	15.8 %	38	203	241
> CLIHarness	24.2 %	23	72	95
> AlertSystem	53.8 %	35	30	65
> PumpController	56.6 %	47	36	83
> DataLogger	57.9 %	70	51	121
▼ SmartWell	65.8 %	375	195	570
▶ tickSeconds(double)	22.2 %	8	28	36
▶ sampleSensors()	52.2 %	24	22	46
▶ emit()	52.9 %	9	8	17
▶ changeWaterLevel(double)	92.5 %	37	3	40
▶ applyEnvironment(double, double, d	96.3 %	129	5	134
▶ clamp(double, double, double)	100.0 %	6	0	6
▶ SmartWell(double, double, double, C	100.0 %	71	0	71
▶ buildContext()	100.0 %	39	0	39
▶ managePumpControl()	100.0 %	52	0	52
> c114_SmartWellSystemEx	67.9 %	74	35	109
> Sensor	69.0 %	20	9	29
> Str	80.0 %	12	3	15
> WaterLevelSensor	81.2 %	52	12	64
> WaterQualitySensor	91.1 %	112	11	123
> TemperatureSensor	95.0 %	38	2	40

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c114_SmartWellSystemEx.java	33.0 %	31	63	94
> AlertSystem	0.0 %	0	2	2
> ReplayEngine	0.0 %	0	8	8
> SimulationEngine	0.0 %	0	8	8
> DataLogger	25.0 %	2	6	8
▼ SmartWell	46.4 %	13	15	28
tickSeconds(double)	25.0 %	1	3	4
applyEnvironment(double, double, d	50.0 %	3	3	6
changeWaterLevel(double)	50.0 %	2	2	4
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	2	2	4
managePumpControl()	66.7 %	4	2	6
clamp(double, double, double)		0	0	0
SmartWell(double, double, double, C		0	0	0
buildContext()		0	0	0
> c114_SmartWellSystemEx	50.0 %	3	3	6
> TemperatureSensor	50.0 %	3	3	6
> WaterLevelSensor	50.0 %	2	2	4
> WaterQualitySensor	50.0 %	5	5	10
> PumpController	75.0 %	3	1	4
> CLIHarness		0	0	0
> Sensor		0	0	0
> SmartWellContext		0	0	0
> Str		0	0	0

5. Fuzz testing instruction coverage: 75%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c114_SmartWellSystemEx.java	32.0 %	945	2,007	2,952
> SmartWellContext	7.3 %	3	38	41
> ReplayEngine	7.4 %	9	113	122
> SimulationEngine	15.8 %	38	203	241
> CLIHarness	24.2 %	23	72	95
> PumpController	51.8 %	43	40	83
> AlertSystem	53.8 %	35	30	65
> DataLogger	57.9 %	70	51	121
▼ SmartWell	63.2 %	360	210	570
tickSeconds(double)	22.2 %	8	28	36
sampleSensors()	52.2 %	24	22	46
emit()	52.9 %	9	8	17
managePumpControl()	75.0 %	39	13	52
changeWaterLevel(double)	87.5 %	35	5	40
applyEnvironment(double, double, d	96.3 %	129	5	134
clamp(double, double, double)	100.0 %	6	0	6
SmartWell(double, double, double, C	100.0 %	71	0	71
buildContext()	100.0 %	39	0	39
> c114_SmartWellSystemEx	67.9 %	74	35	109
> Sensor	69.0 %	20	9	29
> Str	80.0 %	12	3	15
> WaterLevelSensor	81.2 %	52	12	64
> WaterQualitySensor	91.1 %	112	11	123
> TemperatureSensor	95.0 %	38	2	40

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c114_SmartWellSystemEx.java	37.2 %	35	59	94
> AlertSystem	0.0 %	0	2	2
> ReplayEngine	0.0 %	0	8	8
> SimulationEngine	0.0 %	0	8	8
> DataLogger	25.0 %	2	6	8
> c114_SmartWellSystemEx	50.0 %	3	3	6
> TemperatureSensor	50.0 %	3	3	6
> WaterLevelSensor	50.0 %	2	2	4
> WaterQualitySensor	50.0 %	5	5	10
▼ SmartWell	57.1 %	16	12	28
▶ tickSeconds(double)	25.0 %	1	3	4
▶ applyEnvironment(double, double, d	50.0 %	3	3	6
▪ emit()	50.0 %	1	1	2
▪ sampleSensors()	50.0 %	2	2	4
▶ changeWaterLevel(double)	75.0 %	3	1	4
▪ managePumpControl()	100.0 %	6	0	6
▪ clamp(double, double, double)		0	0	0
▶ SmartWell(double, double, double, L		0	0	0
▶ buildContext()		0	0	0
> PumpController	100.0 %	4	0	4
> CLIHarness		0	0	0
> Sensor		0	0	0
> SmartWellContext		0	0	0
> Str		0	0	0

7. Z3 instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c114_SmartWellSystemEx.java	32.7 %	964	1,988	2,952
> SmartWellContext	7.3 %	3	38	41
> ReplayEngine	7.4 %	9	113	122
> SimulationEngine	15.8 %	38	203	241
> CLIHarness	24.2 %	23	72	95
> AlertSystem	53.8 %	35	30	65
> PumpController	56.6 %	47	36	83
> DataLogger	57.9 %	70	51	121
▼ SmartWell	65.8 %	375	195	570
▶ tickSeconds(double)	22.2 %	8	28	36
▪ sampleSensors()	52.2 %	24	22	46
▪ emit()	52.9 %	9	8	17
▶ changeWaterLevel(double)	92.5 %	37	3	40
▶ applyEnvironment(double, double, d	96.3 %	129	5	134
▪ clamp(double, double, double)	100.0 %	6	0	6
▶ SmartWell(double, double, double, L	100.0 %	71	0	71
▶ buildContext()	100.0 %	39	0	39
▪ managePumpControl()	100.0 %	52	0	52
> c114_SmartWellSystemEx	67.9 %	74	35	109
> Sensor	69.0 %	20	9	29
> Str	80.0 %	12	3	15
> WaterLevelSensor	81.2 %	52	12	64
> WaterQualitySensor	91.1 %	112	11	123
> TemperatureSensor	95.0 %	38	2	40

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c114_SmartWellSystemEx.java	37.2 %	35	59	94
> AlertSystem	0.0 %	0	2	2
> ReplayEngine	0.0 %	0	8	8
> SimulationEngine	0.0 %	0	8	8
> DataLogger	25.0 %	2	6	8
> c114_SmartWellSystemEx	50.0 %	3	3	6
> TemperatureSensor	50.0 %	3	3	6
> WaterLevelSensor	50.0 %	2	2	4
> WaterQualitySensor	50.0 %	5	5	10
▼ SmartWell	57.1 %	16	12	28
tickSeconds(double)	25.0 %	1	3	4
applyEnvironment(double, double, d	50.0 %	3	3	6
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	2	2	4
changeWaterLevel(double)	75.0 %	3	1	4
managePumpControl()	100.0 %	6	0	6
clamp(double, double, double)		0	0	0
SmartWell(double, double, double, L		0	0	0
buildContext()		0	0	0
> PumpController	100.0 %	4	0	4
> CLIHarness		0	0	0
> Sensor		0	0	0
> SmartWellContext		0	0	0
> Str		0	0	0

9. Unimproved V-method instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c114_SmartWellSystemEx.java	32.7 %	964	1,988	2,952
> SmartWellContext	7.3 %	3	38	41
> ReplayEngine	7.4 %	9	113	122
> SimulationEngine	15.8 %	38	203	241
> CLIHarness	24.2 %	23	72	95
> AlertSystem	53.8 %	35	30	65
> PumpController	56.6 %	47	36	83
> DataLogger	57.9 %	70	51	121
▼ SmartWell	65.8 %	375	195	570
tickSeconds(double)	22.2 %	8	28	36
sampleSensors()	52.2 %	24	22	46
emit()	52.9 %	9	8	17
changeWaterLevel(double)	92.5 %	37	3	40
applyEnvironment(double, double, d	96.3 %	129	5	134
clamp(double, double, double)	100.0 %	6	0	6
SmartWell(double, double, double, L	100.0 %	71	0	71
buildContext()	100.0 %	39	0	39
managePumpControl()	100.0 %	52	0	52
> c114_SmartWellSystemEx	67.9 %	74	35	109
> Sensor	69.0 %	20	9	29
> Str	80.0 %	12	3	15
> WaterLevelSensor	81.2 %	52	12	64
> WaterQualitySensor	91.1 %	112	11	123
> TemperatureSensor	95.0 %	38	2	40